

subagent_model_tiering.md — §11.4.141 Measure M2 (route mechanical subagents to a cheaper model)

Field	Value
Revision	1
Last modified	2026-06-09T00:00:00Z
Status	active
Authority	constitution submodule, universal (§11.4.17). Measure M2 of §11.4.141 — the biggest non-cache win.

What this is. The rule + mechanism for routing **MECHANICAL, NON-JUDGMENT** subagent work (grep / search / status-probe / simple-edit / parse / doc-export / file-presence / read-only device probe) to a **cheaper / smaller model** (Haiku-class), while the **strong model owns ALL judgment** — every PASS/FAIL verdict, every fix design / root-cause (§11.4.102), every code review (§11.4.125), every demotion-evidence decision (§11.4.7). **The cheap model NEVER emits a verdict**, so §11.4.50 determinism and the anti-bluff covenant are untouched — quality cannot degrade. Subagents also persist large output to a file and return a short pointer instead of an inline 350-520K-token transcript.

0. The bright line — what may and may NOT be tiered down

TIER-DOWN (mechanical, safe → Haiku-class)	STRONG-MODEL-ONLY (judgment → NEVER tier down)
code/log search, grep, glob	any PASS / FAIL / WARN / SKIP verdict

TIER-DOWN (mechanical, safe → Haiku-class)	STRONG-MODEL-ONLY (judgment → NEVER tier down)
status probes (dumpsys, getprop, /proc, /sys reads)	fix design / root-cause (§11.4.102)
doc export / regen (pandoc / weasyprint)	code review (§11.4.125)
file-presence / path checks	demotion-evidence judgment (§11.4.7)
parse / lint / format-only edits	captured-evidence interpretation (is this audio glitchy?)
markdown sync, mechanical renames	architectural / blast-radius analysis (§11.4.92)
read-only device probes (§11.4.96 SAFE catalogue)	anything producing a release-affecting decision

The allowlist is expressed as a **task class**, never as project package names — so it is universal. If a task could **decide** anything (does this PASS? is this the root cause? is this safe to ship?), it is strong-model-only, full stop.

1. The registry (data, not code)

actions/subagent_tiering.yaml is the single source of truth mapping a task class → its model tier. A consuming project ships its own (or inherits this default by reference, like the §11.4.80 codegraph_* scripts). Adding a task class = adding one row — no code change.

```
# actions/subagent_tiering.yaml (excerpt – see the file for the
# full default set)
mechanical_model: haiku # cheaper/smaller tier for the
# mechanical allowlist
judgment_model: inherit # strong tier: the conductor's own
# model (opus-class)
classes:
- { name: code_search, tier: mechanical }
- { name: status_probe, tier: mechanical }
- { name: doc_export, tier: mechanical }
- { name: pass_fail_verdict, tier: judgment } # NEVER
# mechanical
- { name: fix_design, tier: judgment }
- { name: code_review, tier: judgment }
```

scripts/subagent_tier.sh <task-class> reads this registry and prints the model tier name (mechanical / judgment) and the resolved model id, so a dispatcher can pick the model without hardcoding.

It **refuses** (exit 1) any attempt to route a judgment-tier class to the mechanical model (the mechanical enforcement that makes the bright line real).

2. Per-agent mechanism

Agent	Mechanical (tier-down)	Judgment (strong)
Claude Code	subagent model: haiku (alias) on the mechanical allowlist	model: inherit / opus for everything that decides
Gemini / Qwen	Gemini (cheap discovery) for the mechanical pass	Qwen / the strong model for synthesis + verdicts
Cursor / Aider	per-request small-model selection for mechanical legwork	strong model for review + decisions

Output-to-file. A mechanical subagent writes its large output to a file under the project's own artifact dir (e.g. qa-results/ / recordings/, already git-ignored per §11.4.128) and returns a **short pointer** (path + 1-line summary). This is *more* faithful, not less — the full evidence is on disk per §11.4.5/§11.4.69; only the conductor's context shrinks.

3. The safety argument (why M2 cannot degrade quality)

1. The strong model still **owns every decision and every verdict** — the cheap model never emits a PASS/FAIL, so the §11.4 anti-bluff covenant is untouched.
2. The mechanical allowlist is a **task class**, mechanically checkable — subagent_tier.sh refuses to tier a judgment class, so the bright line cannot be crossed by accident.
3. Output-to-file preserves the **full** evidence on disk (§11.4.5/ §11.4.69); only the inline transcript shrinks.
4. Bounded by §12.6 (60% memory) + the §11.4.58 / §11.4.103 stream caps — M2 changes which model does the legwork, not how many streams run.

A judgment-tier task routed to the mechanical model is the violation the §1.1 paired mutation exploits (mutate subagent_tier.sh to tier down pass_fail_verdict → the gate FAILs).

4. Decoupling / reuse (§11.4.28)

The allowlist is a task-class taxonomy (universal); the artifact dirs are *configured* (the project supplies them), never hardcoded. A different project inherits M2 by shipping its own actions/subagent_tiering.yaml and pointing its dispatcher at subagent_tier.sh.

Cross-references

- §11.4.141 (M2) · §11.4.50 (determinism — cheap model never decides) · §11.4.102 (fix-design strong-only) · §11.4.125 (code-review strong-only) · §11.4.7 (demotion strong-only) · §11.4.5/§11.4.69 (output-to-file preserves evidence) · §11.4.128 (artifact dirs) · §12.6 / §11.4.58 / §11.4.103 (stream + memory caps) · §1.1 (tier-down-a-verdict paired mutation).
- actions/subagent_tiering.yaml (the registry) · scripts/subagent_tier.sh (the resolver).